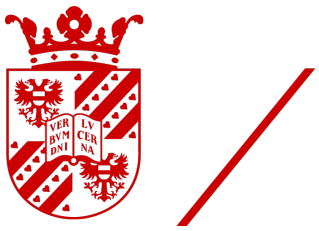

university of groningen

INTRODUCTION TO OPTIMIZATION

Computational Exercise I: Report

Authors:
Briana- Mihaela Balea
Elisaveta Bobrova

Student number:
S5310423
S5215250

Introduction

In this report, we evaluate the performance of several optimization algorithms applied to the *Least Squares Problem*. Each algorithm operates on a matrix A, a vector b, an initial guess vector x_0, a tolerance (10^{-4}) and a number of maximum iterations (10^5). **Constant Step Gradient Descent** utilizes a fixed step size for each iteration,. The **Exact Line Search** method dynamically adjusts the step size, while the Gradient Descent with **Armijo's Rule**, a variant of Line Search, selects the step size by employing a backtracking strategy based on the Armijo condition. The **Conjugate Gradient** method, specifically designed for quadratic functions, iterates along conjugate directions. The **Quasi-Newton** method approximates the Hessian matrix using the BFGS formula. Both **Polyak's Heavy Ball** and **Nesterov's Momentum** methods incorporate momentum, with Nesterov's method predicting future gradients. Lastly, **Incremental Stochastic Gradient Descent** updates the solution based on a single randomly selected sample at each iteration.

Comparison of Optimization Methods

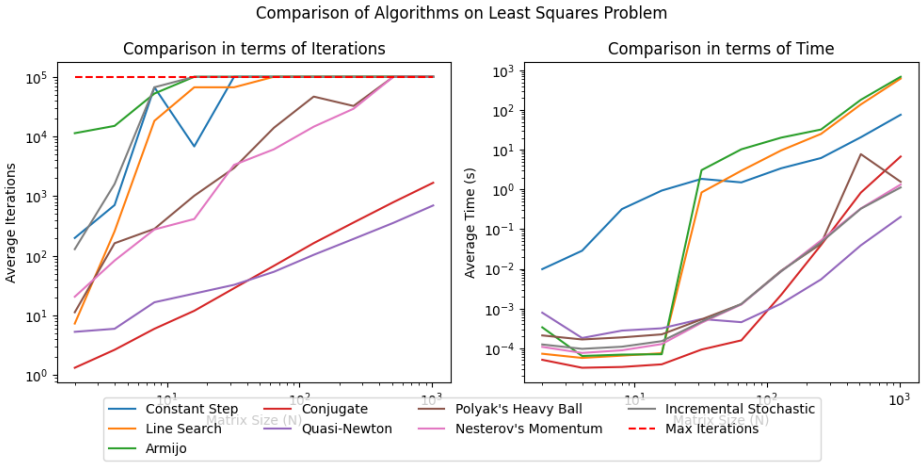

Figure 1: Comparison of the algorithms in terms of iteration count (left) and runtime (right) for the Least Squares problem.

In terms of **iterations**, Quasi-Newton and Conjugate Gradient methods perform the best, converging with significantly fewer iterations due to their use of second-order information (Conjugate Gradient is between Newton's method and the method of Steepest Descent). Conjugate Gradient is particularly efficient for strongly convex quadratic functions, theoretically converging in at most N iterations for matrices of size $N \times N$. Momentum-based methods, namely Polyak's Heavy Ball and Nesterov's Momentum, also perform efficiently due to the acceleration applied to a standard gradient descent but take more iterations than Quasi-Newton and Conjugate Gradient. Meanwhile, Constant Step GD, Exact Line Search, and GD with Armijo's Rule only rely on gradient information and require significantly more iterations, particularly as matrix sizes increase. Finally, although Incremental SGD has a reduced computational costs per iteration, it requires many iterations overall to converge.

In terms of **runtime**, Quasi-Newton and Conjugate Gradient are fast as they converge in the least iterations and the algorithms stop (though per iteration Quasi-Newton becomes slower for large matrix sizes due to an increased cost of approximating the Hessian). Polyak's Heavy Ball and Nesterov's Momentum both stop converging in less that 10^5 iterations only for very large matrix sizes, hence why their runtimes are also efficient overall. Incremental SGD is a very fast algorithm due to its low per-iteration cost, hence why its runtime is also among the lowest ones, despite the many iterations. Lastly, since the Constant Step, Exact Line Search and Armijo Rule methods all require more than 10^5 iterations for convergence after some small matrix size, their runtimes are all significantly higher. Per iteration, Exact Line Search and Armijo's Rule are more efficient than Constant Step which explains the big difference in runtime for smaller matrix sizes.

A Appendix: Python Codes for Methods

This appendix contains the Python implementations of the various optimization methods discussed in this report. Each method has been implemented as a function, which takes as input the matrix A, vector b, initial guess x_0, a tolerance ϵ, and a maximum number of iterations.

A.1 Gradient Descent with Constant Step Size

The function below uses constant step size $\gamma = \max \sigma(A^T A)$ for all iterations.

```
    def constant_step(A, b, x0, tol, max_it):
    # Pre-Computations
    ATA = A.T @ A
    ATb = A.T @ b
    df = lambda x: ATA @ x - ATb

    eigvals, eigvecs = np.linalg.eig(A.T @ A)
    L = max(eigvals)

    # Variables to be set
    gamma = 1 / L

    # Perform the iterations
    for it in range(1, max_it + 1):
        x0-= gamma * df(x0)
        if np.linalg.norm(df(x0)) < tol:
            break

    # Return the final value
    final = x0 if it < max_it else np.nan
    flag = it < max_it
    return final, flag, it
```

A.2 Gradient Descent with Exact Line Search

The following function performs Gradient Descent with exact line search to determine the optimal step size γ at each iteration.

```
def line_search(A, b, x0, tol, max_it):
    # Pre-Computations
    ATA = A.T @ A
    ATb = A.T @ b
    df = lambda x: ATA @ x - ATb

    # Perform the iterations
    for it in range(1, max_it + 1):
        v = -A @ df(x0)
        gamma = (v.T @ (-A @x0 + b))/ (v.T @ v)
        x0-= gamma * df(x0)
        if np.linalg.norm(df(x0)) < tol:
            break

    # Return the final value
    final = x0 if it < max_it else np.nan
    flag = it < max_it
    return final, flag, it
```

A.3 Gradient Descent with Armijo Rule

This function implements Gradient Descent with the Armijo rule to adaptively adjust the step size γ based on the decrease in the objective function.

```
def armijo(A, b, x0, tol, max_it):
    # Pre-Computations
    ATA = A.T @ A
    ATb = A.T @ b
    df = lambda x: ATA @ x - ATb
    f = lambda x: 0.5 * np.linalg.norm(A @ x - b) ** 2

    # Variables to be set
    beta = 1e-2
    sigma = 1e-4

    # Perform the iterations
    for it in range(1, max_it + 1):
        gamma = 1e-2
        while f(x0 - gamma * df(x0)) > f(x0) - sigma * gamma * ( df(x0).T @ df(x0) ):
            gamma = beta * gamma
        x0 -= gamma * df(x0)
        if np.linalg.norm(df(x0)) < tol:
            break

    # Return the final value
    final = x0 if it < max_it else np.nan
    flag = it < max_it
    return final, flag, it
```

A.4 Conjugate Gradient Descent

This function implements the Conjugate Gradient Method for solving the least squares problem.

```
def conjugate(A, b, x0, tol, max_it):
    #Pre-computations
    ATA= A.T @ A
    ATb = A.T @ b
    df = lambda x: ATA @ x -ATb

    #Variables to be set
    gamma = 0
    p = -df(x0)
    r = df(x0)

    #Perform iterations
    for it in range(1, max_it + 1):
        alpha = - (p.T @ r) / (p.T @ ATA @ p)
        x0 += alpha * p
        if np.linalg.norm(df(x0)) < tol:
          break
        r_new = r + alpha * ATA @ p
        gamma = (r_new.T @ r_new) / (r.T @ r)
        p = - r_new + gamma * p
        r = r_new

    final = x0 if it < max_it else np.nan
    flag = it < max_it
    return final, flag, it
```

A.5 Quasi-Newton Method

The Quasi-Newton method is implemented in the following function, which approximates the Hessian and updates iteratively. Notice that if the function sp.optimize.line_search does not converge we pick a small α.

```
def quasi(A, b, x0, tol, max_it):
    # Pre-Computations
    ATA = A.T @ A
    ATb = A.T @ b
    df = lambda x: ATA @ x - ATb

    #Modified to handle both 1-D and 2-D arrays
    df_flat = lambda x: (A.T @ (A @ x.reshape(-1, 1) - b)).flatten()
    #Modified to handle both 1-D and 2-D arrays
    f = lambda x: 0.5 * np.linalg.norm(A @ x.reshape(-1, 1) - b) ** 2
    n, n = A.shape
    x0 = x0.reshape((n, 1))

    # Variables to be set
    I = np.eye(len(x0))
    #Initial approximation of the inverse Hessian D0
    D = I

    for it in range(1, max_it + 1):
        grad = df(x0)
        u = -D @ grad  # Search direction

        #Use x0.flatten() to ensure a 1-D array is being used for line_search
        alpha = sp.optimize.line_search(f, df_flat, x0.flatten(), u.flatten())[0]
        if alpha is None:  #If line search fails, pick a small step size
            alpha = 1e-4

        s = alpha * u
        x_new = x0 + s
        g = df(x_new) - grad
        p = 1 / (g.T @ s)[0,0]
        D = (I - p * s @ g.T) @ D @ (I - p * g @ s.T) + p * s @ s.T

        x0 = x_new
        if np.linalg.norm(grad) < tol:
            break

    final = x0 if it < max_it else np.nan
    flag = it < max_it
    return final, flag, it
```

A.6 Polyak's Heavy Ball Method

The following function implements Polyak's Heavy Ball method, which uses fixed momentum parameter for accelerated convergence.

```
def polyak(A, b, x0, tol=1e-4, max_it=10**5):
    # Pre-Computations
    ATA = A.T @ A
    ATb = A.T @ b
    df = lambda x: ATA @ x - ATb

    # Compute Lipschitz constant L and strong convexity constant mu
    eigvals = np.linalg.eigvals(A.T @ A)
```

```
    L = np.max(eigvals)  # Lipschitz constant
    mu = np.min(eigvals)  # Strong convexity constant

    # Optimal parameters for Polyak's Heavy Ball
    gamma = 4 / (np.sqrt(L) + np.sqrt(mu))**2
    theta = ((np.sqrt(L) - np.sqrt(mu)) / (np.sqrt(L) + np.sqrt(mu)))**2

    # Variables to be set
    x_prev = x0
    grad = A.T @ (A @ x0 - b)

    for it in range(1, max_it + 1):
      x = x0 + theta * (x0 - x_prev) - gamma * df(x0)
      x_prev = x0
      x0 = x
      if np.linalg.norm(df(x0)) < tol:
        break

    # Return the final value
    final = x0 if it < max_it else np.nan
    flag = it < max_it
    return final, flag, it
```

A.7 Nesterov's Momentum

This function implements Nesterov's Accelerated Gradient method which makes use of an intermediate value y.

```
def nesterov(A, b, x0, tol, max_it):
    #Pre-computations
    ATA = A.T @ A
    ATb = A.T @ b
    df = lambda x: ATA @ x - ATb

    eigvals = np.linalg.eigvals(ATA)
    L = np.max(eigvals)  # Largest eigenvalue (Lipschitz constant)
    mu = np.min(eigvals)  # Smallest eigenvalue (strong convexity constant)

    #Step size
    gamma = 1 / L

    # Acceleration factor (theta) based on the given formula
    theta = (np.sqrt(L) - np.sqrt(mu)) / (np.sqrt(L) + np.sqrt(mu))

    # Initialization
    x_prev = x0     # Previous iterate (for momentum)

    for it in range(1, max_it + 1):
        y = x0 + theta * (x0 - x_prev)

        # Update the next point using gradient descent
        x_new = y - gamma * df(y)

        # Update previous and current points for the next iteration
        x_prev = x0
        x0 = x_new
        # Check for convergence
        if np.linalg.norm(df(x0)) < tol:
          break

```

```
    # Return the final value
    final = x0 if it < max_it else np.nan
    flag = it < max_it
    return final, flag, it
```

A.8 Incremental Stochastic Gradient Descent

The Incremental Stochastic Gradient Descent method which updates the solution based on a randomly selected sample is implemented as follows:

```
def stochastic(A, b, x0, tol=1e-4, max_it=10**5):
    #Pre-computations
    n, n = A.shape
    ATA = A.T @ A
    ATb = A.T @ b
    df = lambda x: ATA @ x - ATb

    # Perform iterations
    for it in range(1, max_it+1):
      alpha= 1 / (it + 1)

      # Randomly select a row
      i = np.random.randint(0, n)

      # Get the i-th row of A and the corresponding b_i
      a_i = A[i, :].reshape((1, n))
      b_i = b[i]

      # Compute the stochastic gradient w.r.t. a single sample
      gradient = a_i.T @ (a_i @ x0 - b_i)

      x0 = x0 - alpha * gradient

      # Check for convergence
      if np.linalg.norm(df(x0)) < tol:
        break

    final = x0 if it < max_it else np.nan
    flag = it < max_it
    return final, flag, it
```

B Appendix: Python Codes for Plots

This section contains all the codes necessary for plotting the comparison of algorithms in A for the *Least Squares Problem*.

B.1 Average iterations

Begin by setting the global parameters.

```
    # Set global parameters
    np.random.seed(42)
    tol = 1e-4
    max_it = 10 ** 5
```

Now, compute the average iterations for each method.

B.1.1 Gradient Descent with Constant Step size

```

    # Empty array to store the average number of iterations
    avg_its_constant_step = np.empty(10)
    flag_constant_step = np.empty(3, dtype=bool)

    # Generate each average
    for k in tqdm(range(1, 11)):
        time.sleep(1)
        # Set parameters and generate matrices
        N = 2 ** k

    # Compute average over 3 runs
    total_its_constant_step = 0
    for i in range(3):
        A = np.random.random((N, N))
        b = np.random.random((N, 1))
        x0 = np.zeros((N, 1))

        _, flag_constant_step[i], total_its_constant_step   = constant_step(A, b, x0, tol, max_it)
        total_its_constant_step += total_its_constant_step

    if flag_constant_step[0] == False and flag_constant_step[1] == False and flag_constant_step[2] ==
    False:
        avg_its_constant_step[k-1:] = 10 ** 5
        break

    avg_its_constant_step[k-1] = total_its_constant_step / 3
```

B.1.2 Gradient Descent with Exact Line Search

```
    # Empty array to store the average number of iterations
    avg_its_line_search = np.empty(10)
    flag_line_search = np.empty(3, dtype=bool)

    # Generate each average
    for k in tqdm(range(1, 11)):
    time.sleep(1)
    # Set parameters and generate matrices
    N = 2 ** k

    # Compute average over 3 runs
    total_its_line_search = 0
    for i in range(3):
        A = np.random.random((N, N))
        b = np.random.random((N, 1))
        x0 = np.zeros((N, 1))

        _, flag_line_search[i], total_its_line_search = line_search(A, b, x0, tol, max_it)
        total_its_line_search += total_its_line_search

    if flag_line_search[0] == False and flag_line_search[1] == False and flag_line_search[2] == False:
        avg_its_line_search[k-1:] = 10 ** 5
        break

    avg_its_line_search[k-1] = total_its_line_search / 3
```

B.1.3 Gradient Descent with Armijo Rule

```
# Empty array to store the average number of iterations
avg_its_armijo = np.empty(10)
flag_armijo = np.empty(3, dtype=bool)

# Generate each average
for k in tqdm(range(1, 11)):
    time.sleep(1)
    # Set parameters and generate matrices
    N = 2 ** k

# Compute average over 3 runs
total_its_armijo = 0
for i in range(3):
    A = np.random.random((N, N))
    b = np.random.random((N, 1))
    x0 = np.zeros((N, 1))

    _, flag_armijo[i], total_its_armijo = armijo(A, b, x0, tol, max_it)
    total_its_armijo += total_its_armijo

if flag_armijo[0] == False and flag_armijo[1] == False and flag_armijo[2] == False:
    avg_its_armijo[k-1:] = 10 ** 5
    break

avg_its_armijo[k-1] = total_its_armijo / 3
```

B.1.4 Conjugate Gradient Descent

```
# Empty array to store the average number of iterations
avg_its_conjugate = np.empty(10)
flag_conjugate = np.empty(3, dtype=bool)

# Generate each average
for k in tqdm(range(1, 11)):
    time.sleep(1)
    # Set parameters and generate matrices
    N = 2 ** k

# Compute average over 3 runs
total_its_conjugate = 0
for i in range(3):
    A = np.random.random((N, N))
    b = np.random.random((N, 1))
    x0 = np.zeros((N, 1))

    _, flag_conjugate[i], total_its_conjugate = conjugate(A, b, x0, tol, max_it)
    total_its_conjugate += total_its_conjugate

if flag_conjugate[0] == False and flag_conjugate[1] == False and flag_armijo[2] == False:
    avg_its_conjugate[k-1:] = 10 ** 5
    break

avg_its_conjugate[k-1] = total_its_conjugate / 3
```

B.1.5 Quasi-Newton Method

```
    # Empty array to store the average number of iterations
    avg_its_quasi = np.empty(10)
    flag_quasi = np.empty(3, dtype=bool)

    # Generate each average
    for k in tqdm(range(1, 11)):
        time.sleep(1)
        # Set parameters and generate matrices
        N = 2 ** k

    # Compute average over 3 runs
    total_its_quasi = 0
    for i in range(3):
        A = np.random.random((N, N))
        b = np.random.random((N, 1))
        x0 = np.zeros((N, 1))

        _, flag_quasi[i], total_its_quasi = quasi(A, b, x0, tol, max_it)
        total_its_quasi += total_its_quasi

    if flag_quasi[0] == False and flag_quasi[1]== False and flag_quasi[2]== False:
        avg_its_quasi[k-1:] = 10 ** 5
        break

    avg_its_quasi[k-1] = total_its_quasi / 3
```

B.1.6 Polyak's Heavy Ball Method

```
    # Empty array to store the average number of iterations
    avg_its_polyak = np.empty(10)
    flag_polyak = np.empty(3, dtype=bool)

    # Generate each average
    for k in tqdm(range(1, 11)):
        time.sleep(1)
        # Set parameters and generate matrices
        N = 2 ** k

    # Compute average over 3 runs
    total_its_polyak = 0
    for i in range(3):
        A = np.random.random((N, N))
        b = np.random.random((N, 1))
        x0 = np.zeros((N, 1))

        _, flag_polyak[i], total_its_polyak= polyak(A, b, x0, tol, max_it)
        total_its_polyak += total_its_polyak

    if flag_polyak[0] == False and flag_polyak[1] == False and flag_polyak[2] == False:
        avg_its_polyak[k-1:] = 10 ** 5
        break

    avg_its_polyak[k-1] = total_its_polyak / 3
```

B.1.7 Nesterov's Momentum

```
    # Empty array to store the average number of iterations
    avg_its_nesterov = np.empty(10)
    flag_nesterov = np.empty(3, dtype=bool)

    # Generate each average
    for k in tqdm(range(1, 11)):
        time.sleep(1)
        # Set parameters and generate matrices
        N = 2 ** k

    # Compute average over 3 runs
    total_its_nesterov = 0
    for i in range(3):
        A = np.random.random((N, N))
        b = np.random.random((N, 1))
        x0 = np.zeros((N, 1))

        _, flag_nesterov[i], total_its_nesterov = nesterov(A, b, x0, tol, max_it)
        total_its_nesterov += total_its_nesterov

    if flag_nesterov[0] == False and flag_nesterov[1] == False and flag_nesterov[2] == False:
        avg_its_nesterov[k-1:] = 10 ** 5
        break

    avg_its_nesterov[k-1] = total_its_nesterov / 3
```

B.1.8 Incremental Stochastic Gradient Descent

```
    # Empty array to store the average number of iterations
    avg_its_stochastic = np.empty(10)
    flag_stochastic = np.empty(3, dtype=bool)
    # Generate each average
    for k in tqdm(range(1, 11)):
        time.sleep(1)
        # Set parameters and generate matrices
        N = 2 ** k

    # Compute average over 3 runs
    total_its_stochastic = 0
    for i in range(3):
        A = np.random.random((N, N))
        b = np.random.random((N, 1))
        x0 = np.zeros((N, 1))

        _, flag_stochastic[i], total_its_stochastic = stochastic(A, b, x0, tol, max_it)
        total_its_stochastic += total_its_stochastic

    if flag_stochastic[0] == False and flag_stochastic[1] == False and flag_stochastic[2] == False:
        avg_its_stochastic[k-1:] = 10 ** 5
        break

    avg_its_stochastic[k-1] = total_its_stochastic / 3
```

B.1.9 The Plot

Finally, plot all algorithms in the same figure.

```
    # Plot the curves
    plt.figure(figsize = (8,6))

    max_it = 10 ** 5
    plt.loglog(2 ** np.linspace(1, 10, 10), [max_it] * 10, "r--", label="Max Iterations")

    plt.loglog(2 ** np.linspace(1, 10, 10), avg_its_constant_step, label="Constant Step")
    plt.loglog(2 ** np.linspace(1, 10, 10), avg_its_line_search, label="Line Search")
    plt.loglog(2 ** np.linspace(1, 10, 10), avg_its_armijo, label="Armijo")
    plt.loglog(2 ** np.linspace(1, 10, 10), avg_its_conjugate, label="Conjugate")
    plt.loglog(2 ** np.linspace(1, 10, 10), avg_its_quasi, label="Quasi-Newton")
    plt.loglog(2 ** np.linspace(1, 10, 10), avg_its_polyak, label="Polyak's Heavy Ball")
    plt.loglog(2 ** np.linspace(1, 10, 10), avg_its_nesterov, label="Nesterov's Momentum")
    plt.loglog(2 ** np.linspace(1, 10, 10), avg_its_stochastic, label="Incremental Stochastic")

    plt.title("Comparison of Algorithms on Least Squares Problem")
    plt.xlabel("N")
    plt.ylabel("Average Iterations")
    plt.legend()
    plt.show()
```

B.2 Average Time

The following code computes the average time for each of the methods.

```
# Empty array to store the average time
avg_time_constant_step = np.empty(10)
avg_time_line_search = np.empty(10)
avg_time_armijo = np.empty(10)
avg_time_conjugate = np.empty(10)
avg_time_quasi = np.empty(10)
avg_time_polyak = np.empty(10)
avg_time_nesterov = np.empty(10)
avg_time_stochastic = np.empty(10)

# Generate each average
for k in tqdm(range(1, 11)):
    time.sleep(1)
    # Set parameters and generate matrices
    N = 2 ** k

    # Compute average over 3 runs
    total_time_constant_step = 0
    total_time_line_search = 0
    total_time_armijo = 0
    total_time_conjugate = 0
    total_time_quasi = 0
    total_time_polyak = 0
    total_time_nesterov = 0
    total_time_stochastic = 0


    for i in range(3):
        A = np.random.random((N, N))
        b = np.random.random((N, 1))
        x0 = np.zeros((N, 1))
```

```
        ATA = A.T @ A
        ATb = A.T @ b
        df = lambda x: ATA @ x - ATb

        start_constant_step = time.time()
        constant_step(A, b, x0, tol, max_it)
        end_constant_step = time.time()
        total_time_constant_step += end_constant_step - start_constant_step

        start_line_search = time.time()
        line_search(A, b, x0, tol, max_it)
        end_line_search = time.time()
        total_time_line_search += end_line_search - start_line_search

        start_armijo = time.time()
        armijo(A, b, x0, tol, max_it)
        end_armijo = time.time()
        total_time_armijo += end_armijo - start_armijo

        start_conjugate = time.time()
        conjugate(A, b, x0, tol, max_it)
        end_conjugate = time.time()
        total_time_conjugate += end_conjugate - start_conjugate

        start_quasi = time.time()
        quasi(A, b, x0, tol, max_it)
        end_quasi = time.time()
        total_time_quasi += end_quasi - start_quasi

        start_polyak = time.time()
        polyak(A, b, x0, tol, max_it)
        end_polyak = time.time()
        total_time_polyak += end_polyak - start_polyak

        start_nesterov = time.time()
        nesterov(A, b, x0, tol, max_it)
        end_nesterov = time.time()
        total_time_nesterov += end_nesterov - start_nesterov

        start_stochastic = time.time()
        stochastic(A, b, x0, tol, max_it)
        end_stochastic = time.time()
        total_time_stochastic += end_stochastic - start_stochastic

    avg_time_constant_step[k-1] = total_time_constant_step / 3
    avg_time_line_search[k-1] = total_time_line_search / 3
    avg_time_armijo[k-1] = total_time_armijo / 3
    avg_time_conjugate[k-1] = total_time_conjugate / 3
    avg_time_quasi[k-1] = total_time_quasi / 3
    avg_time_polyak[k-1] = total_time_polyak / 3
    avg_time_nesterov[k-1] = total_time_nesterov / 3
    avg_time_stochastic[k-1] = total_time_stochastic / 3
```

B.3 Subplots

Here, the plots of matrix size vs average iterations and matrix size vs average time are put side by side.

```
# Create figure
fig, axs = plt.subplots(1, 2, figsize=(10, 5))
fig.suptitle("Comparison of Algorithms on Least Squares Problem")

# Plot the curves
axs[0].loglog(2 ** np.linspace(1, 10, 10), avg_its_constant_step, label="Constant Step")
axs[1].loglog(2 ** np.linspace(1, 10, 10), avg_time_constant_step, label="Constant Step")

axs[0].loglog(2 ** np.linspace(1, 10, 10), avg_its_line_search, label="Line Search")
axs[1].loglog(2 ** np.linspace(1, 10, 10), avg_time_line_search, label="Line Search")

axs[0].loglog(2 ** np.linspace(1, 10, 10), avg_its_armijo, label="Armijo")
axs[1].loglog(2 ** np.linspace(1, 10, 10), avg_time_armijo, label="Armijo")

axs[0].loglog(2 ** np.linspace(1, 10, 10), avg_its_conjugate, label="Conjugate")
axs[1].loglog(2 ** np.linspace(1, 10, 10), avg_time_conjugate, label="Conjugate")

axs[0].loglog(2 ** np.linspace(1, 10, 10), avg_its_quasi, label="Quasi-Newton")
axs[1].loglog(2 ** np.linspace(1, 10, 10), avg_time_quasi, label="Quasi-Newton")

axs[0].loglog(2 ** np.linspace(1, 10, 10), avg_its_polyak, label="Polyak's Heavy Ball")
axs[1].loglog(2 ** np.linspace(1, 10, 10), avg_time_polyak, label="Polyak's Heavy Ball")

axs[0].loglog(2 ** np.linspace(1, 10, 10), avg_its_nesterov, label="Nesterov's Momentum")
axs[1].loglog(2 ** np.linspace(1, 10, 10), avg_time_nesterov, label="Nesterov's Momentum")

axs[0].loglog(2 ** np.linspace(1, 10, 10), avg_its_stochastic, label="Incremental Stochastic")
axs[1].loglog(2 ** np.linspace(1, 10, 10), avg_time_stochastic, label="Incremental Stochastic")

# Plot maximum number of iterations
axs[0].loglog(2 ** np.linspace(1, 10, 10), [max_it] * 10, "r--", label="Max Iterations")

# Plot metadata
axs[0].set_title("Comparison in terms of Iterations")
axs[0].set_xlabel(r"Matrix Size (N)")
axs[0].set_ylabel("Average Iterations")

axs[1].set_title("Comparison in terms of Time")
axs[1].set_xlabel(r"Matrix Size (N)")
axs[1].set_ylabel("Average Time (s)")

handles, labels = axs[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='lower center', ncols=4, bbox_to_anchor=(0.5, 0))
plt.tight_layout()
plt.subplots_adjust(bottom=0.18)

plt.show()
```